1 Update Packages for the Maintenance Manager

1.1 Overview

Update Packages are used to distribute new features via the Maintenance Manager. The following chapter describes the structure of these files.

An update package is an archive file (zip) and can have any name. The file extension *upd* is set by default.

The package can include the following subfolders:

client: MOC client package in **.upd* format (0-n)

➔ You can also deploy these **.upd* files individually.

java: java server package in **.upd* format (0-n)

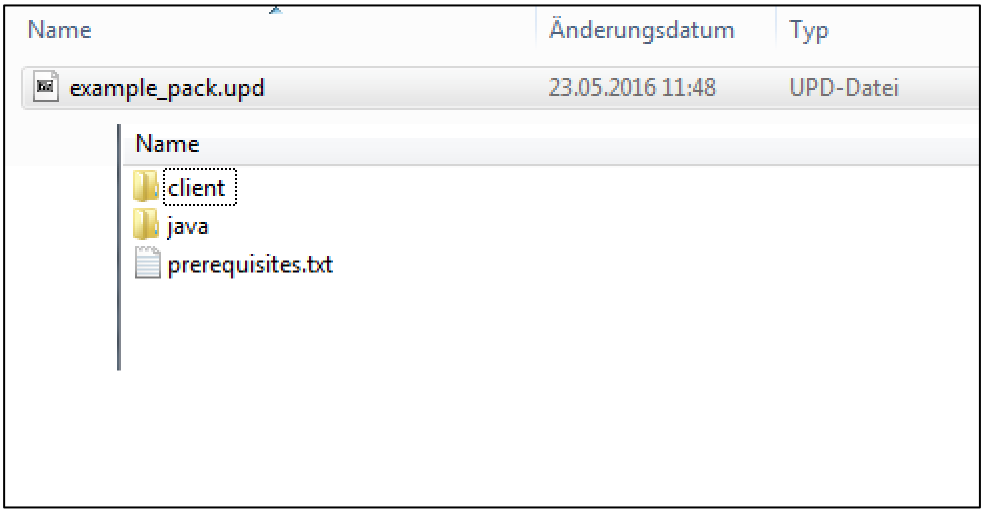
➔ You can also deploy these **.upd* files individually.

server: server packages in **.upd* format (0-n)

➔ You can also deploy these **.upd* files individually.

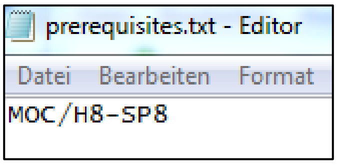
You may find a *prerequisites.txt* file in addition to the folders mentioned above. The *prerequisites.txt* file includes information on the required service pack or hotfix version.

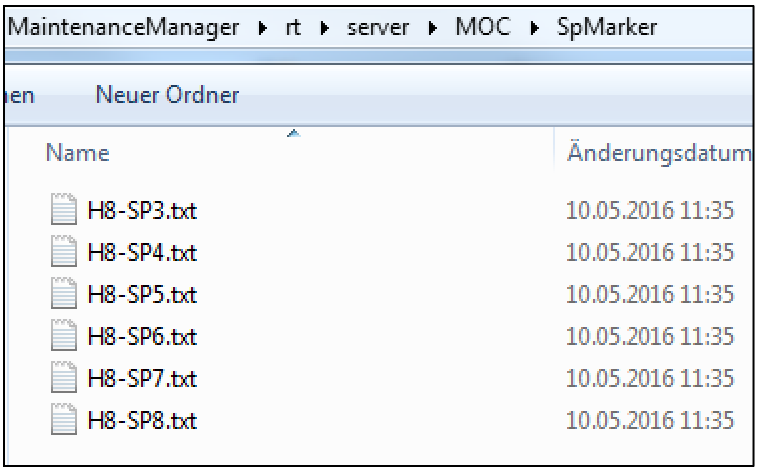
You can install update packages via the *Package Deployment* in the Maintenance Manager.



prerequisites.txt

The file *prerequisites.txt* describes the requirements, i.e. which service pack is needed. You can check in *MaintenanceManager\rt\server\MOC\SpMarker* if the required file exists.





1.2 Black list for MOC updates using Maintenance Manager 2

The update process and update behavior of an MOC installation on a workstation PC have changed if you use Maintenance Manager 2 and the MOC Updater. In contrast to the previous MOC update, where files were only supplemented or updated, the new update process also deletes files.

During the update process, the local MOC installation is compared/synchronized with the reference version in *Maintenance Manager 2.* All files that do not correspond to the server's reference version are overwritten or deleted. This also applies to files created or modified as part of the development of customizations with the MES Development Suite.

To avoid data loss, you can exclude directories or files from the update process. For this purpose, enter the relevant files or directories in an MOC black list. You can only enter files and directories that are located in the MOC main directory!

You can create the black list using any text editor. Save the file as "Blacklist.txt" in the home directory of the MOC Updater `<MOC installation directory>\update\` so that the MOC Updater can process the file.

The file structure must be in JSON format. Enclose each entry in quotation marks. Separate multiple entries via comma.

Example of a *Blacklist.txt* file:

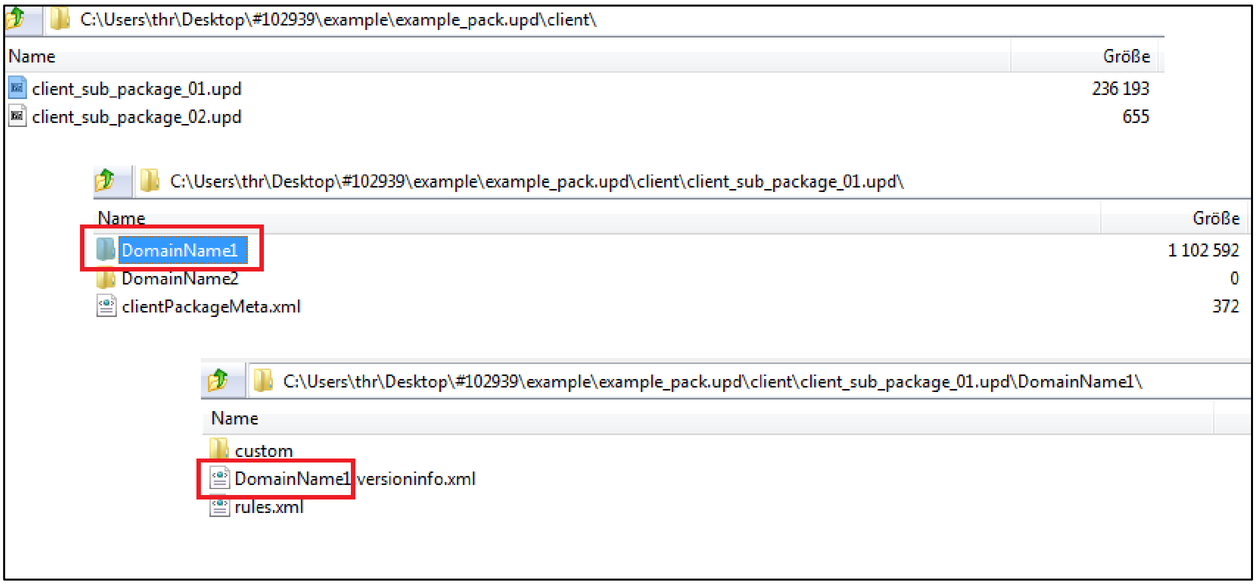
```
{
  "DirectoryBlacklist":
     ["local\\",
      "custom\\",
      "conf\\MOC\\Apps\\"],
  "FileBlacklist":
     ["conf\\DoNotDelete.txt",
      "conf\\DoalsoNotDelete.txt"]
}
```

Important: If you develop own applications in the local scope, enter the directory of the local scope in the black list to prevent the local developments from being deleted by the MOC Updater:

```
{
    "DirectoryBlacklist": ["local\\"]
}
```

1.3 Structure of MOC Client Package

An MOC update package is structured as follows:



clientPackageMeta.xml:

The *clientPackageMeta.xml* in the root directory of the **.upd* folder includes information on the contents of the update package: name of the update package without file extension, description, date of creation, name of the application, 1-n domains.

```xml
<?xml version="1.0" encoding="utf-8"?>
<packageMeta>
  <name>client_sub_package_01</name>
  <description>Description package 01</description>
  <creationDate>2016-05-23T09:30:08</creationDate>
  <applicationName>MOC</appli0cationName>
  <domain version="1.0.CUST.00001">DomainName1</domain>
  <domain version="1.0.CUST.00002">DomainName2</domain>
</packageMeta>
```

#.versioninfo.xml

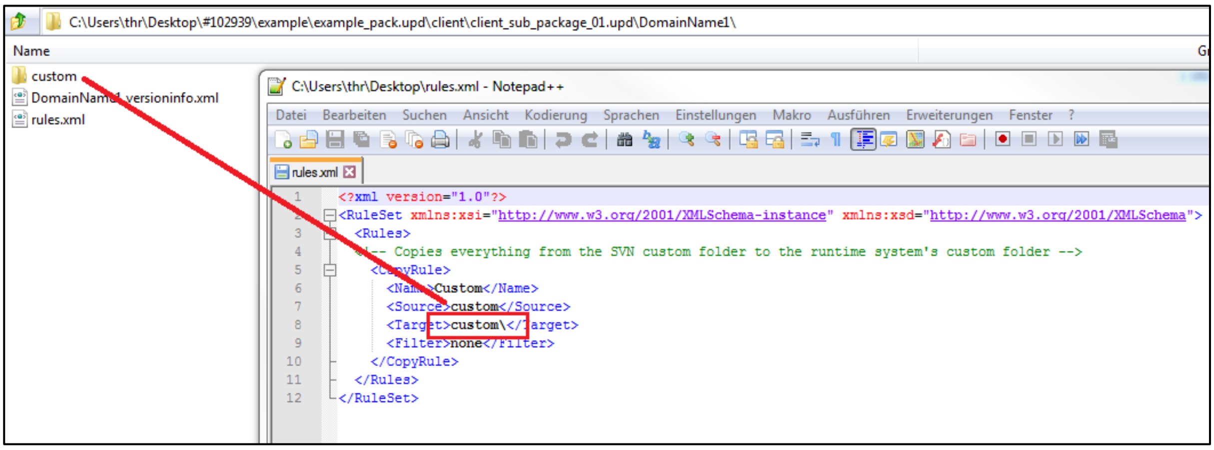
You can find the ***#.versioninfo.xml*** below the higher-level domain. Enter the domain name for the placeholder "#". Enter the correct customer ID and the domain as object ID in this file.

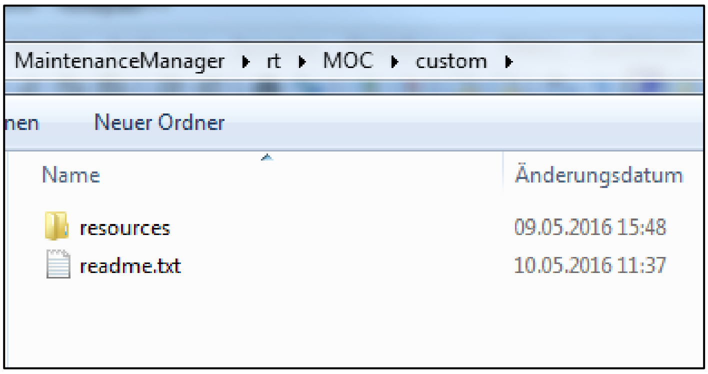
```xml
<?xml version="1.0"?>
<mpdvVersion xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <CustomizedVersion>0</CustomizedVersion>
  <MajorVersion>1</MajorVersion>
  <MinorVersion>0</MinorVersion>
  <Revision>45661</Revision>
  <CustomerId>CUST</CustomerId>
  <ObjectId>DomainName1</ObjectId>
</mpdvVersion>
```

rules.xml:

You can find the *rules.xml* below the higher-level domain. This file includes 1-n copy rules. These rules define which file / which directory (source) is stored in which target directory. Use the filter to select specific files. If you only want to copy *xml* files, enter the following filter: "<filter>*.xml</filter>". This example copies the complete contents of the *custom* folder into the MOC runtime directory. Use the placeholder #SERVER# in the target, to store the files directly in *JHYDRADIR* after activation in the Maintenance Manager.

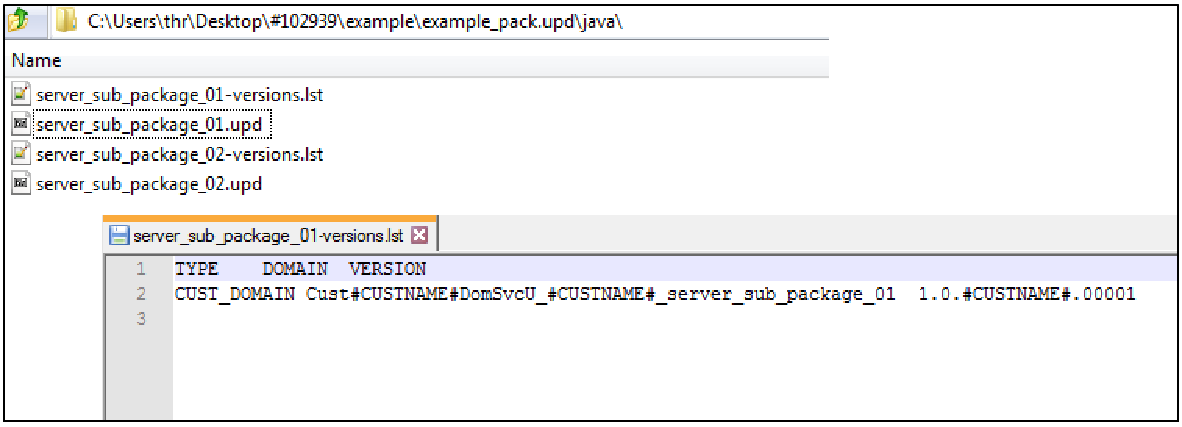
You can find further copy rules in the description of the java server packages.



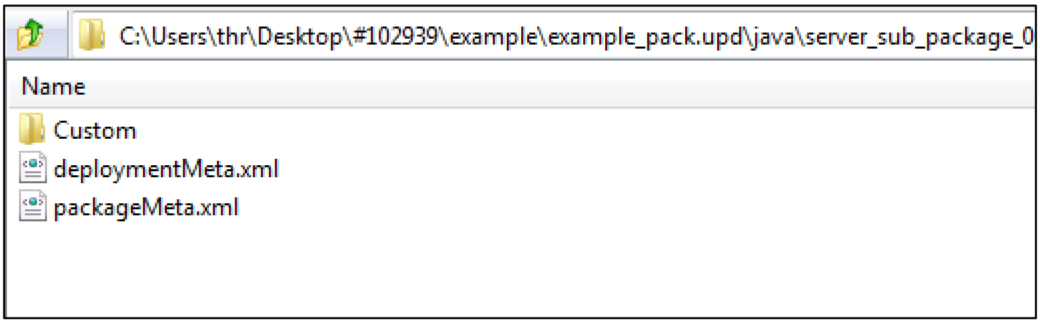


1.4 Structure of Java Server Package

A server update package is structured as follows (the examples mentioned below sometimes include the placeholder #CUSTNAME#; replace this placeholder with the relevant customer name):



.lst files are not relevant for the update package and are created for internal purposes only. This file is not mandatory.



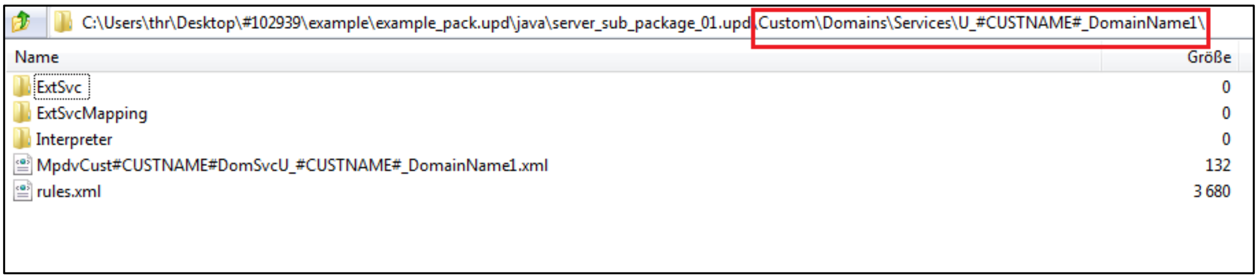
deploymentMeta.xml:

```xml
  <?xml version="1.0" encoding="UTF-8"?>
- <deploymentMeta>
      <warName>MocServices</warName>
      <confContextName>MOC</confContextName>
  </deploymentMeta>
```

packageMeta.xml:

The *packageMeta.xml* in the root directory of the **.upd* folder includes information on the contents of the update package: name of the update package without file extension, description, date of creation, 1-n domains including version, customer, type, path and name.

```xml
  <?xml version="1.0" encoding="UTF-8"?>
- <packageMeta>
      <name>server_sub_package_01</name>
      <applicationName>MOC</applicationName>
      <description>Custom Service for sub package 01</description>
      <creationDate>2016-05-23T11:30:43</creationDate>
      <packageElement version="1.0.#CUSTNAME#.00001" customer="#CUSTNAME#" type="CUST_DOMAIN" path="Custom/Domains/Services/U_#CUSTNAME#_DomainName1"
         name="Cust#CUSTNAME#DomSvcU_#CUSTNAME#_DomainName1"/>
      <packageElement version="1.0.#CUSTNAME#.00002" customer="#CUSTNAME#" type="CUST_DOMAIN" path="Custom/Domains/Services/U_#CUSTNAME#_DomainName2"
         name="Cust#CUSTNAME#DomSvcU_#CUSTNAME#_DomainName2"/>
      <packageElement type="THIRD_PARTY_LIBS" path="ThirdPartyLibs" name="thirdpartylibs"/>
  </packageMeta>
```



MpdvCust#CUSTNAME#DomSvcU_#CUSTNAME#_DomainName1.xml:

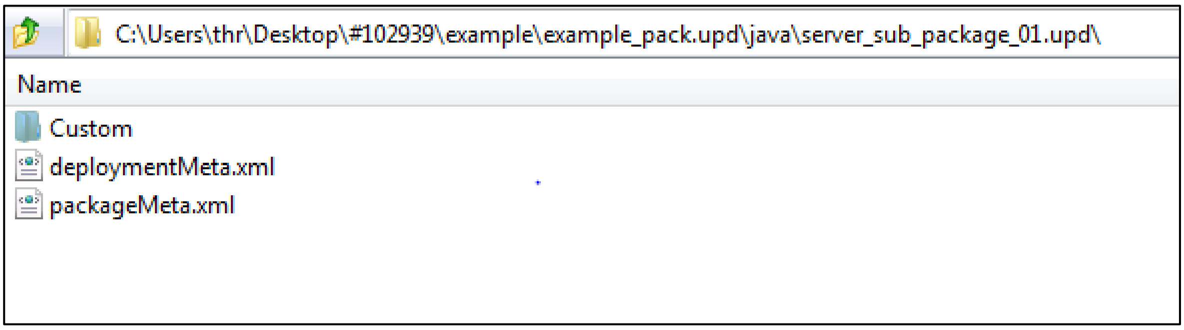
```xml
  <?xml version="1.0" encoding="UTF-8"?>
- <projectMeta>
      <projectType>library</projectType>
      <useInWar>false</useInWar>
  </projectMeta>
```

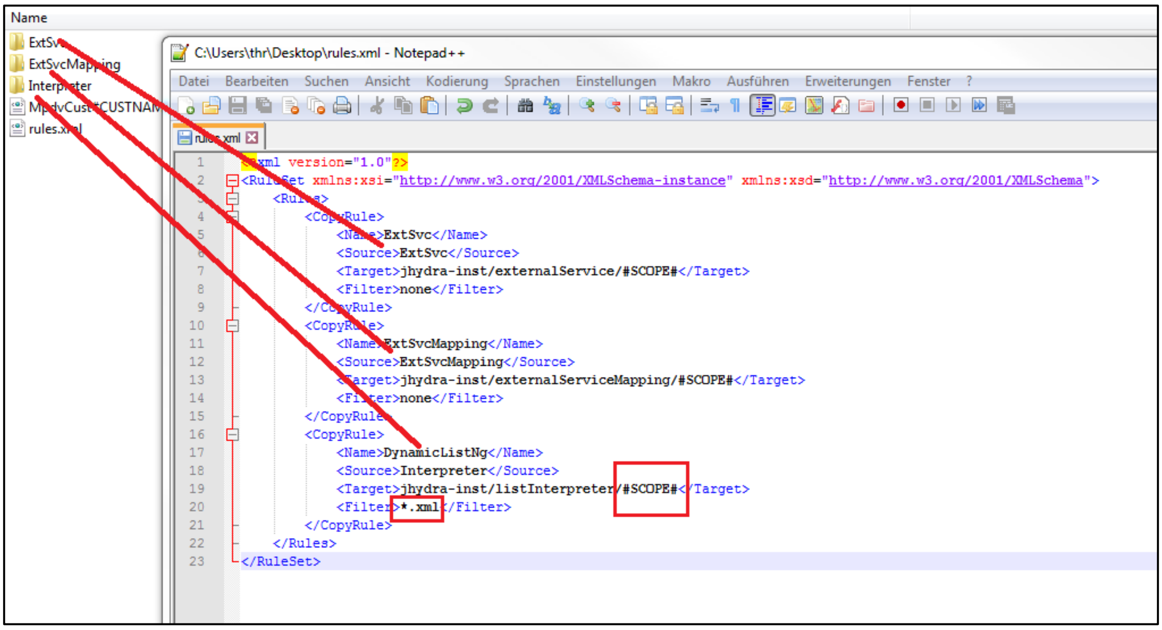
rules.xml:

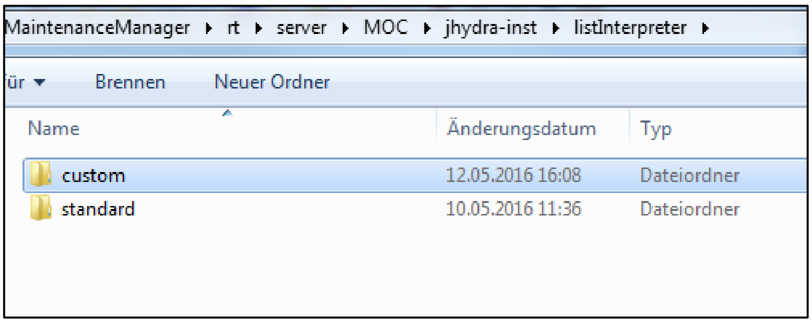
You can find the *rules.xml* below the higher-level domain. This file includes 1-n copy rules. These rules define which file / which directory (source) is stored in which target directory. Use the filter to select specific files. If you only want to copy *xml* files, enter the following filter: "<filter>*.xml</filter>".

This example copies *ExtSvc, ExtSvcMapping* and the folder *Interpreter* to the *JHYDRADIR* (runtime directory of the Maintenance Manager) of the predefined subdirectory. The placeholder #SCOPE# is then replaced with the directory created in the root directory of the update package (e.g. custom, standard, local).

Use the placeholder #CLIENT# in the target to store the files directly in the runtime directory (MOC) after activation in the Maintenance Manager.



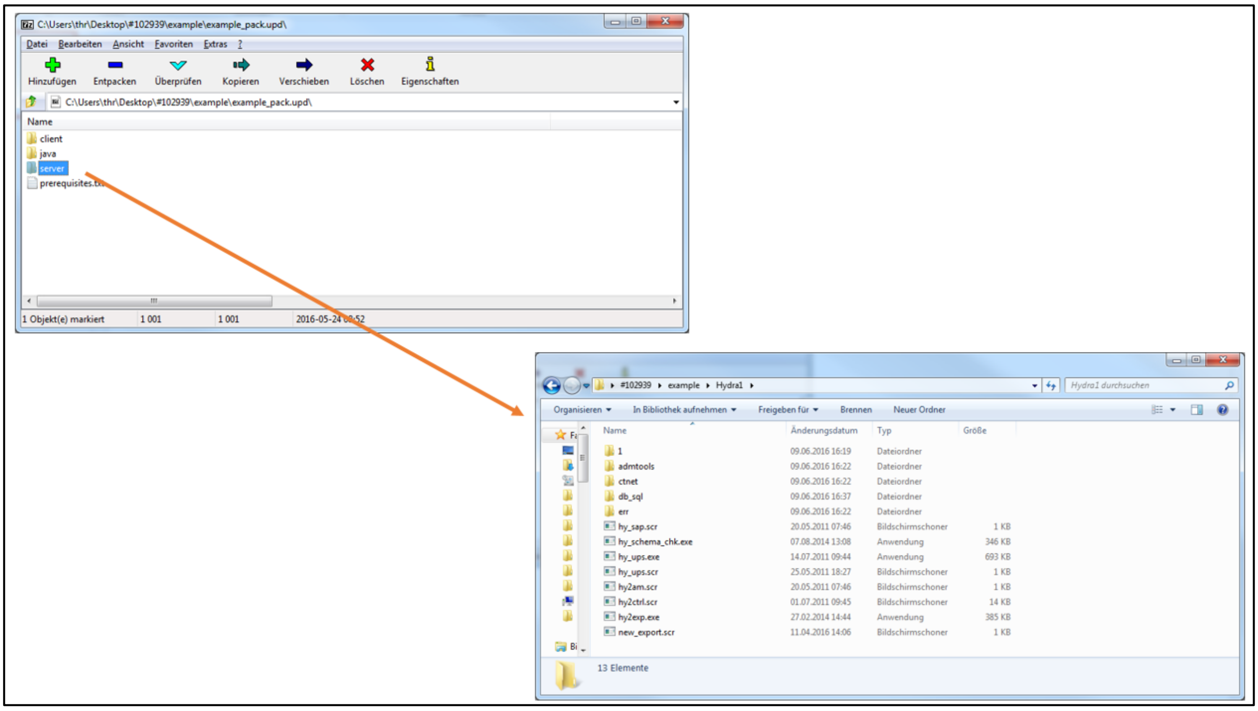




Interpreter copied with custom scope to the runtime directory.

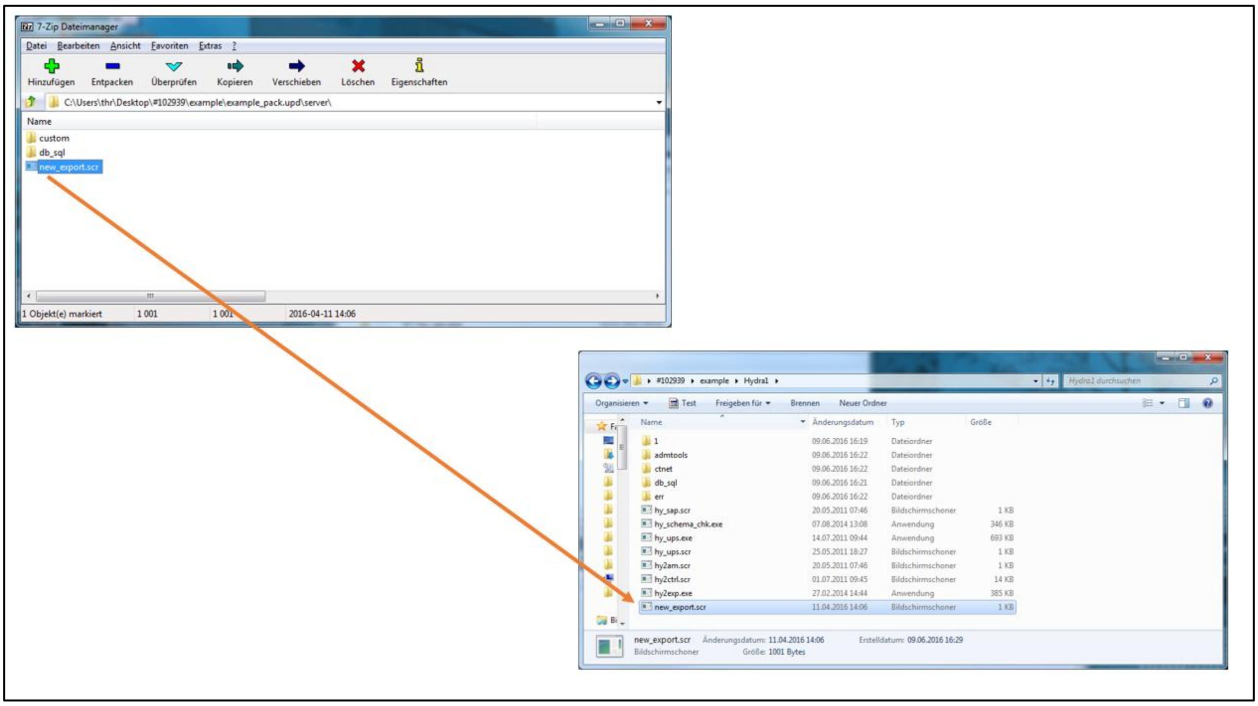
1.5 Structure of Server Package

The system copies the directory structure of the root directory of the update package one-to-one into the HYDRA directory (all subfolders of the *server* directory).

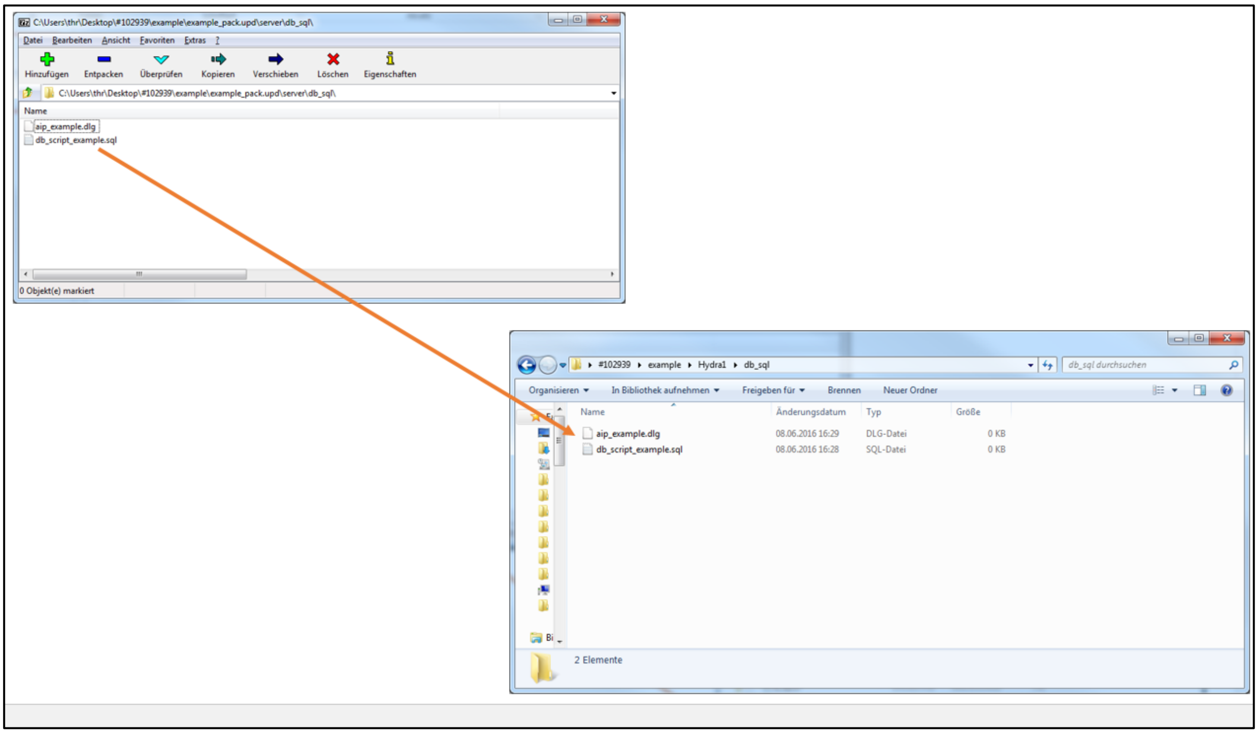


The following example shows a server update package:

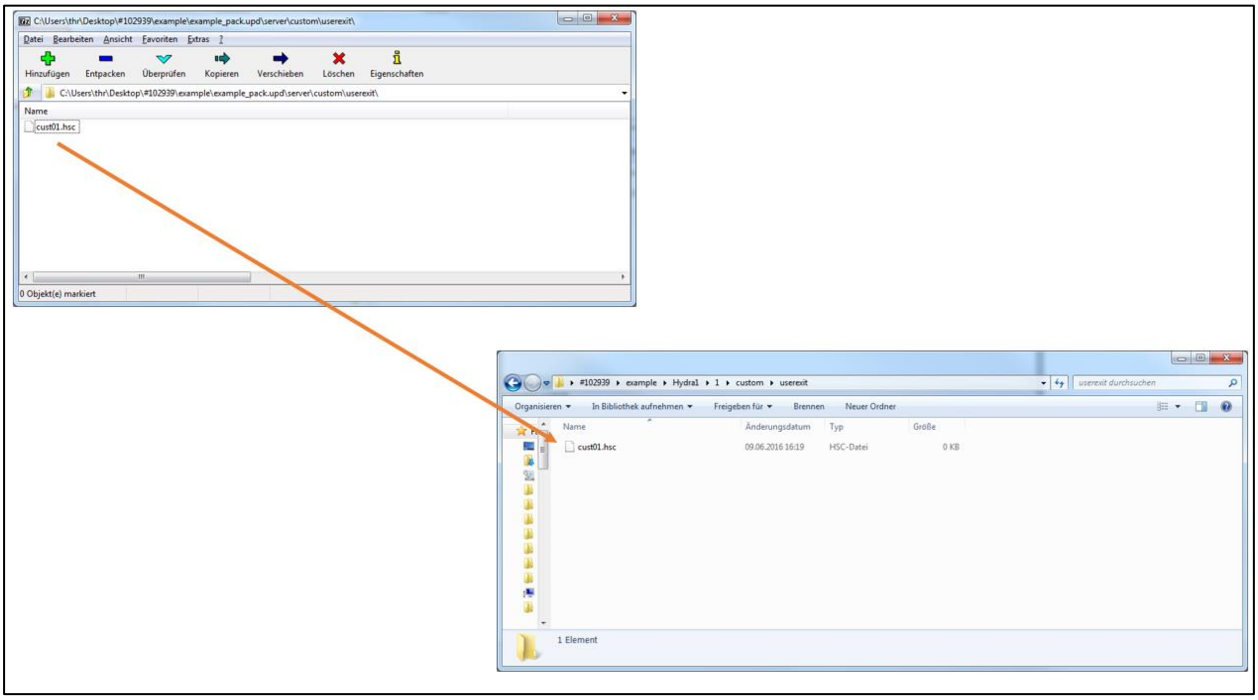
Store server scripts (.scr), programs (.exe/.out), etc. directly in the root directory of the update package. These are stored one-to-one in the HYDRA root directory as described above.



DB patches, SQL scripts, SQL files, dialog files are stored in the subfolder *db_sql*. These are also stored one-to-one (including subfolders) in the HYDRA directory.



Customizations in the form of user exits (terminal scripts, server scripts, SVG files for the upload interface) are stored in the subfolder *custom/userexit*.



Further examples:

Label design: Reports are stored in *custom/reports* (.ll / .qr3 files).

Terminals: Customer-specific INI files are stored in *custom/aip* or *custom/aip2*.

Customer-specific language files are stored in *custom* (hycust.mld).

The directory structure in the update package must be identical to the HYDRA directory structure on the server without leading system number. I.e. in the update package, the path is

/custom/userexit

and not

1/custom/userexit.

If the installation is performed using the Maintenance Manager, the files are automatically copied to the subdirectory with the correct system number.

With update packages of MPDV (e.g. as part of a service pack), files can also be contained in a subdirectory with system number 1 (e.g. 1/custom/userexit). Also these files are automatically copied to the subdirectory with the correct system number if the installation is performed using the Maintenance Manager. This structure is not recommended any more.